

DAY 19



SEMANTIC SEARCH

Overview of Semantic Search and RAG



- There's a lot of research on how to best use language models for search. Three broad categories of these models are dense retrieval, reranking, and RAG. Here is an overview of these three categories that the rest of the chapter will then explain in more detail:

Dense retrieval: Dense retrieval systems rely on the concept of embeddings, the same concept we've encountered in the previous chapters, and turn the search problem into retrieving the nearest neighbors of the search query (after both the query and the documents are converted into embeddings). Figure 8-1 shows how dense retrieval takes a search query, consults its archive of texts, and outputs a set of relevant results.

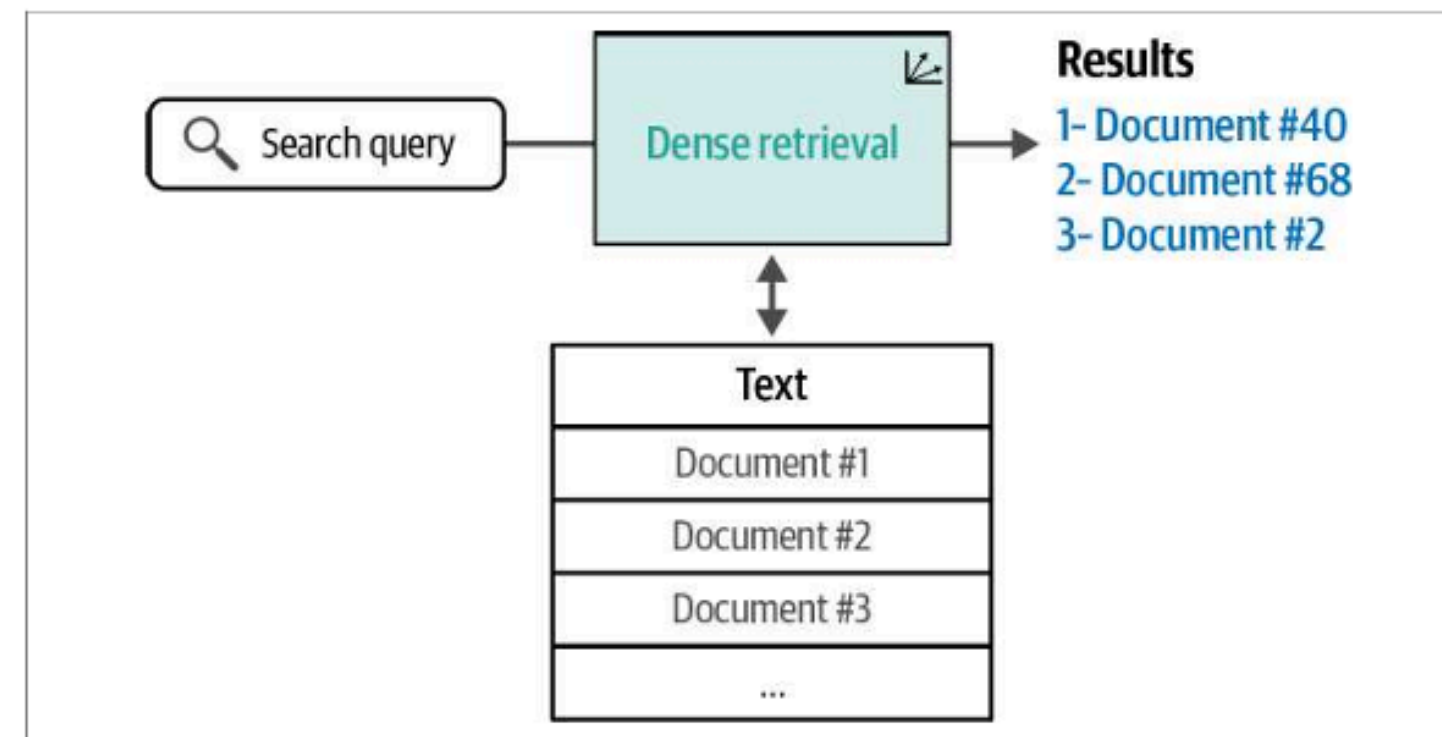


Figure 8-1. Dense retrieval is one of the key types of semantic search, relying on the similarity of text embeddings to retrieve relevant results.



- Reranking: Search systems are often pipelines of multiple steps. A reranking language model is one of these steps and is tasked with scoring the relevance of a subset of results against the query; the order of results is then changed based on these scores. Figure 8-2 shows how rerankers are different from dense retrieval in that they take an additional input: a set of search results from a previous step in the search pipeline.

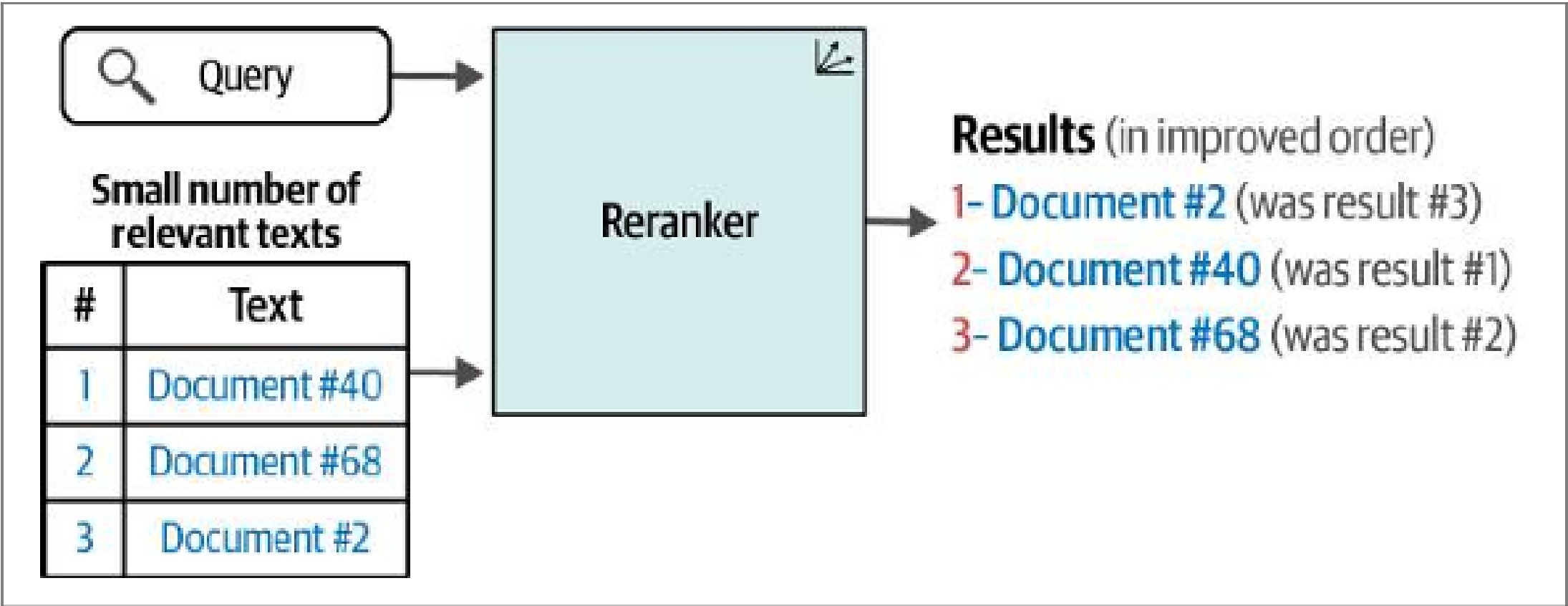


Figure 8-2. Rerankers, the second key type of semantic search, take a search query and a collection of results, and reorder them by relevance, often resulting in vastly improved results.

- RAG :The growing LLM capability of text generation led to a new type of search systems that include a model that generates an answer in response to a query. Figure 8-3 shows an example of such a generative search system. Generative search is a subset of a broader type of category of systems better called RAG systems. These are text generation systems that incorporate search capabilities to reduce hallucinations, increase factuality, and/or ground the generation model on a specific dataset.

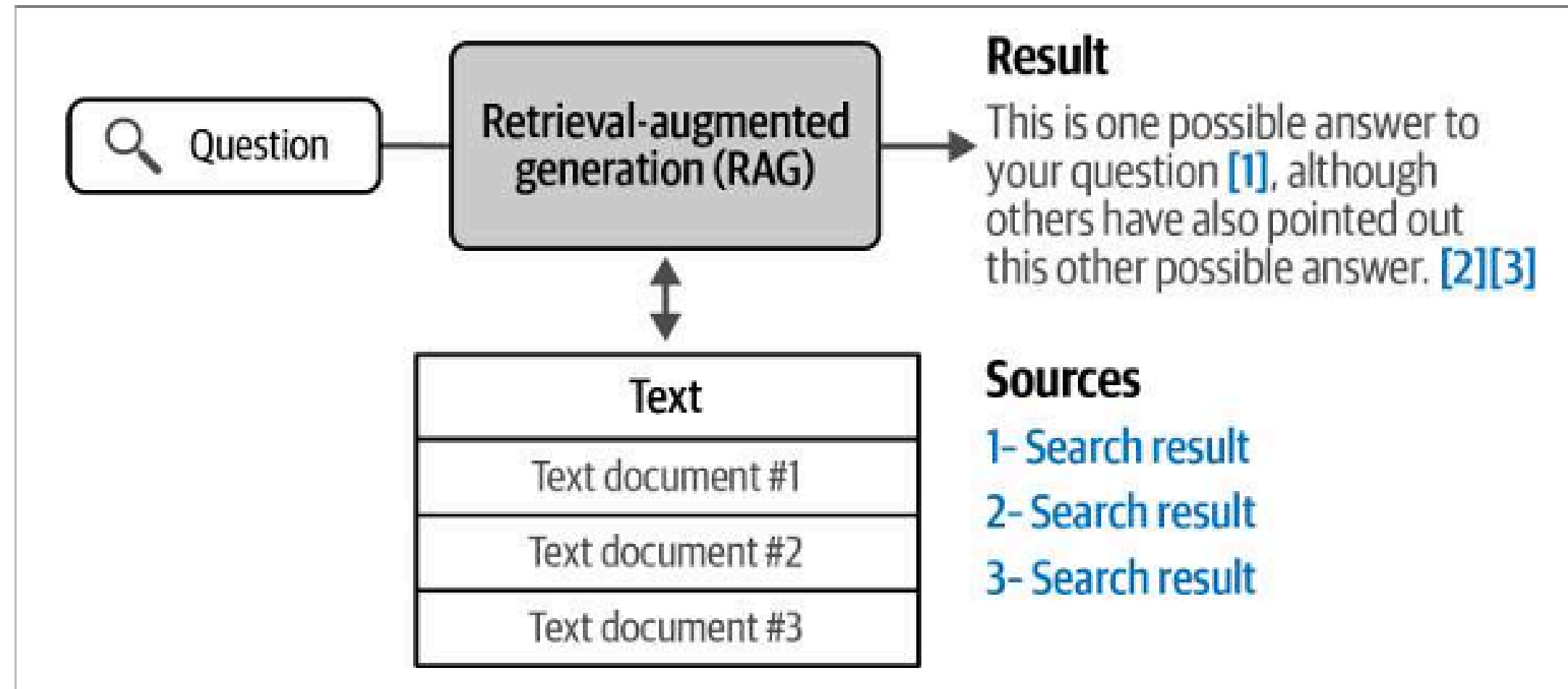


Figure 8-3. A RAG system formulates an answer to a question and (preferably) cites its information sources.

Semantic Search with Language Models



- Let's now dive into more detail on the major categories of systems that can upgrade the search capabilities of our language models. We'll start with dense retrieval and then move on through reranking and RAG.

Dense Retrieval

- Recall that embeddings turn text into numeric representations. Those can be thought of as points in space, as we can see in Figure 8-4. Points that are close together mean that the text they represent is similar. So in this example, text 1 and text 2 are more similar to each other (because they are near each other) than text 3 (because it's farther away).

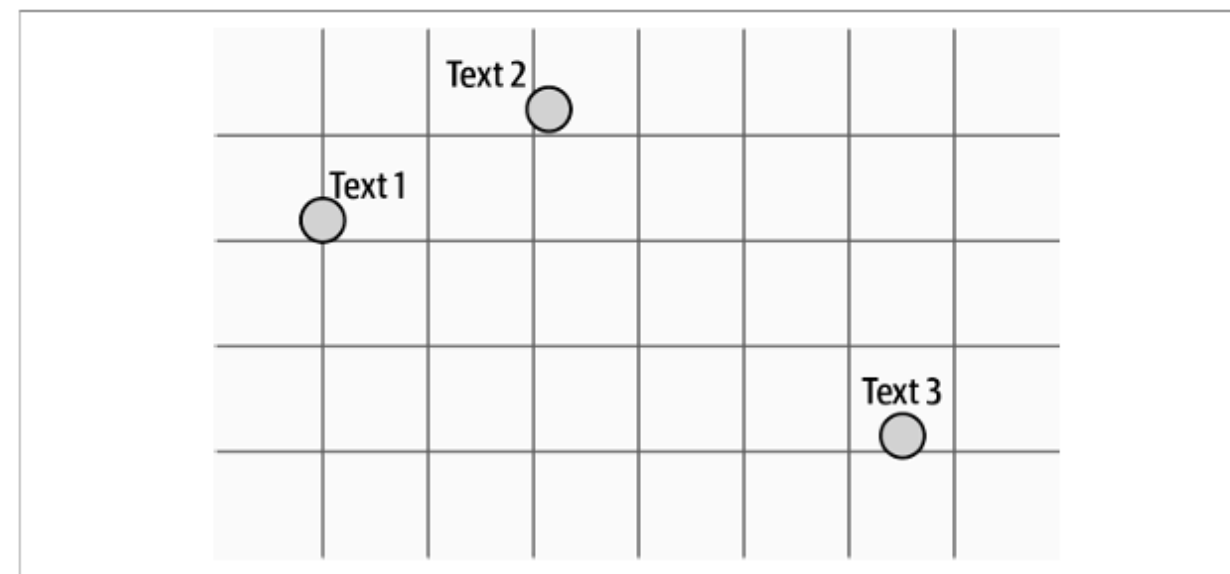


Figure 8-4. The intuition of embeddings: each text is a point and texts with similar meaning are close to each other.

- This is the property that is used to build search systems. In this scenario, when a user enters a search query, we embed the query, thus projecting it into the same space as our text archive. Then we simply find the nearest documents to the query in that space, and those would be the search results (Figure 8-5).

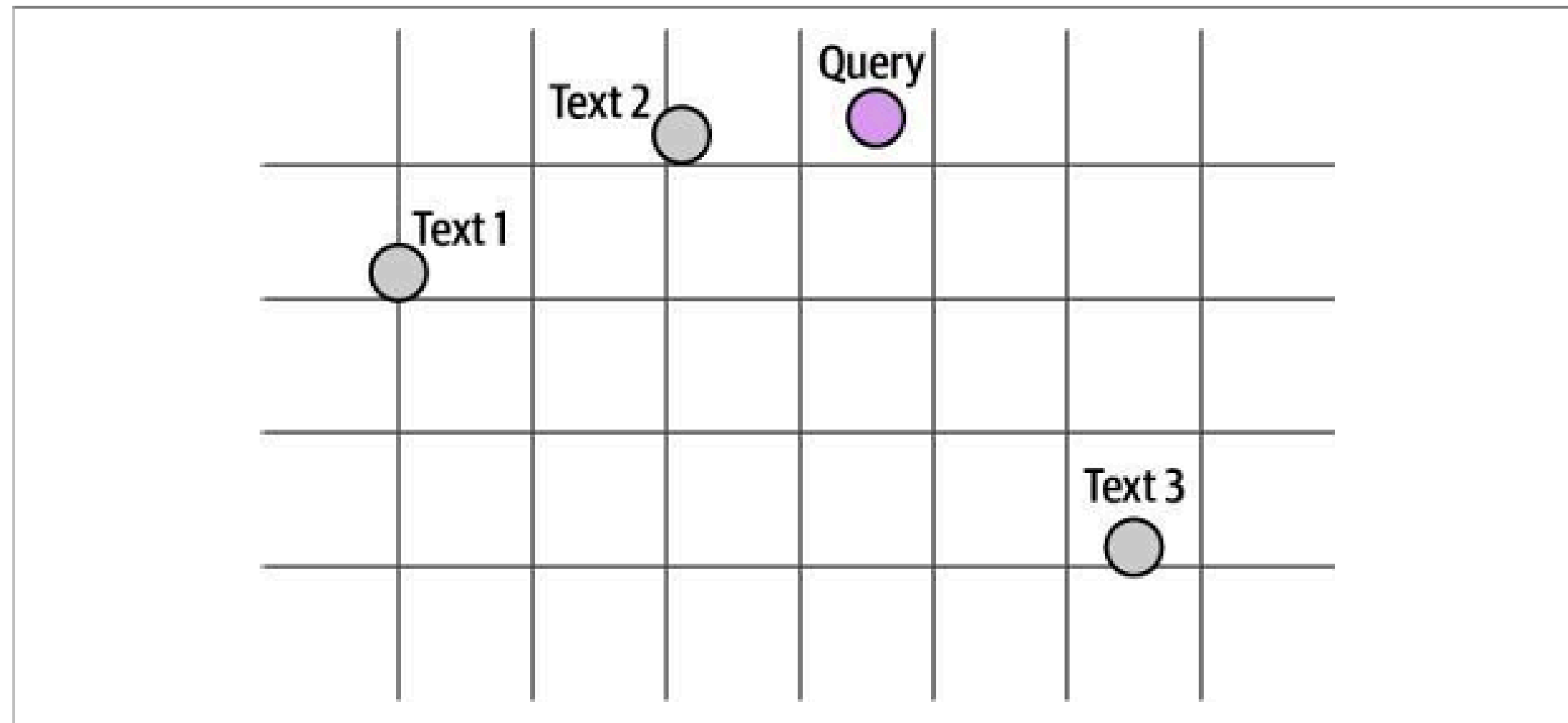


Figure 8-5. Dense retrieval relies on the property that search queries will be close to their relevant results.

- Judging by the distances in Figure 8-5, “text 2” is the best result for this query, followed by “text 1.” Two questions could arise here, however:
- Should text 3 even be returned as a result? That’s a decision for you, the system designer. It’s sometimes desirable to have a max threshold of similarity score to filter out irrelevant results (in case the corpus has no relevant results for the query).
- Are a query and its best result semantically similar? Not always. This is why language models need to be trained on question-answer pairs to become better at retrieval. This process is explained in more detail in Chapter 10.
- Figure 8-6 shows how we chunk a document before proceeding to embed each chunk. Those embedding vectors are then stored in the vector database and are ready for retrieval.

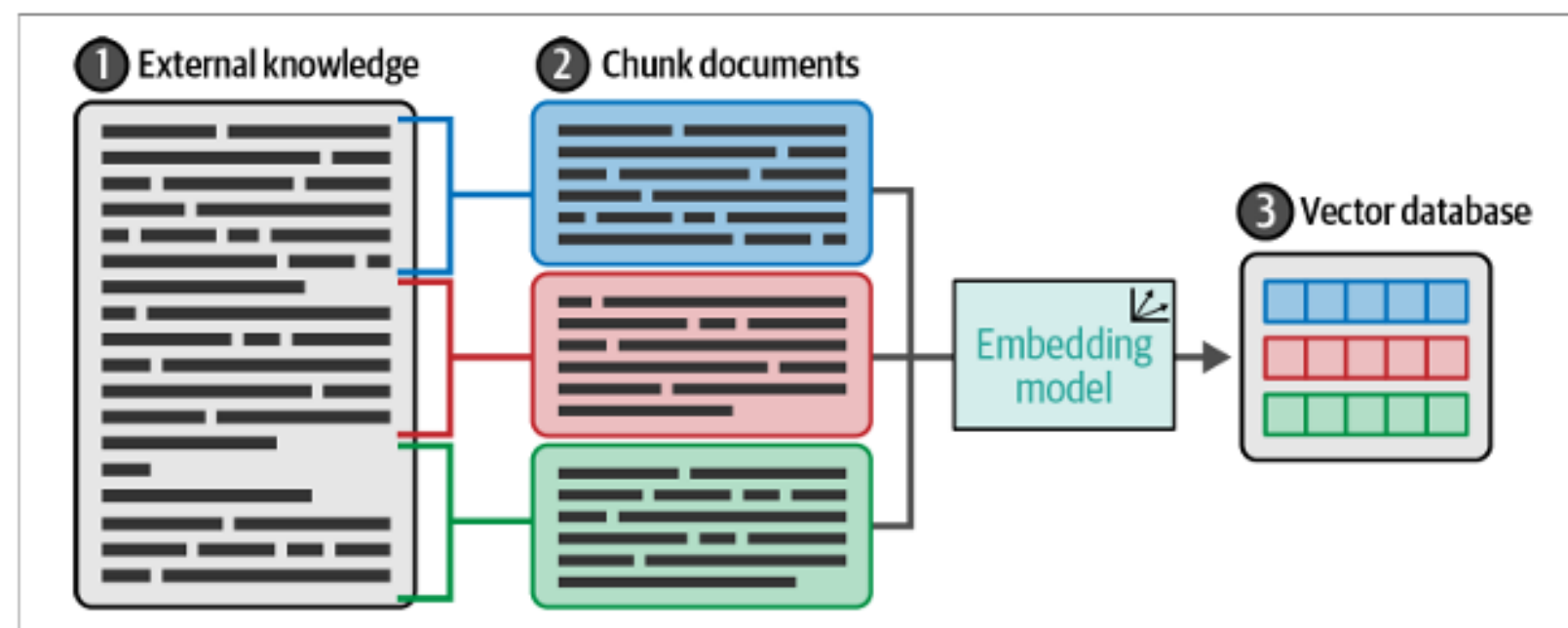


Figure 8-6. Convert some external knowledge base to a vector database. We can then query this vector database for information about the knowledge base.

Reranking



- A lot of organizations have already built search systems. For those organizations, an easier way to incorporate language models is as a final step inside their search pipeline. This step is tasked with changing the order of the search results based on relevance to the search query. This one step can vastly improve search results and it's in fact what Microsoft Bing added to achieve the improvements to search results using BERT-like models. Figure 8-14 shows the structure of a rerank search system serving as the second stage in a two-stage search system.

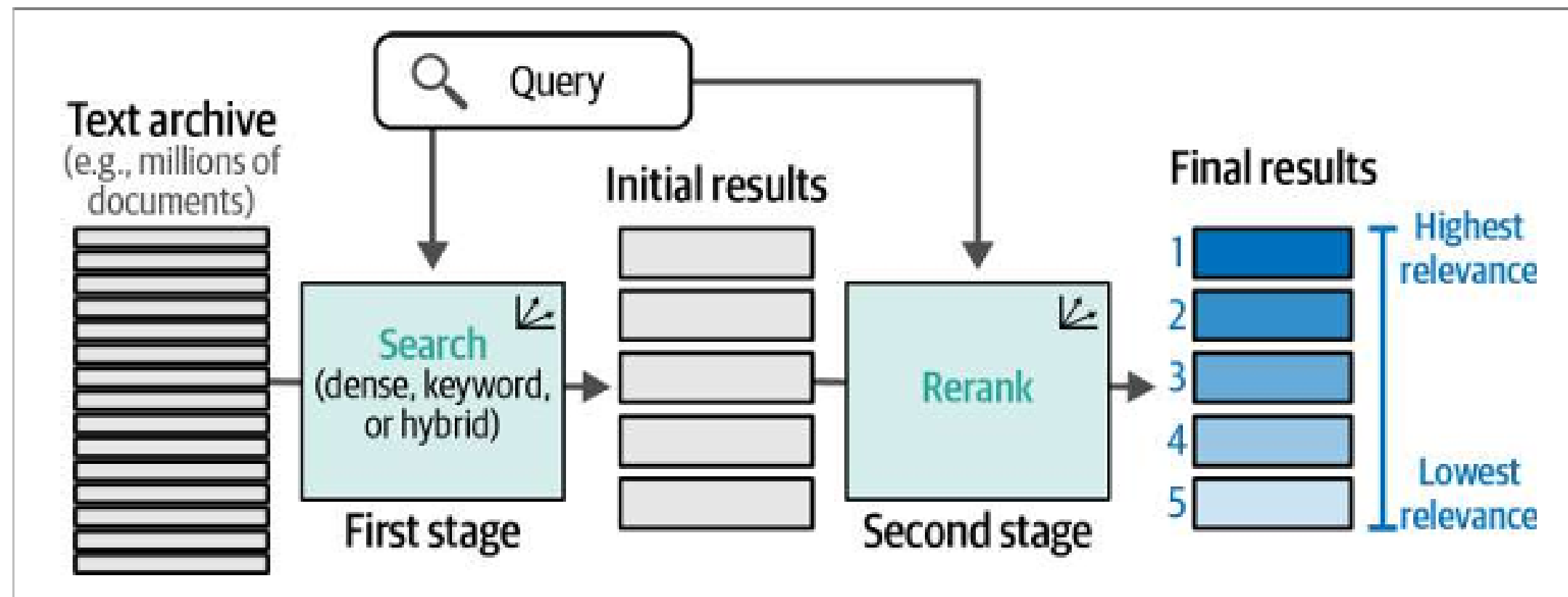


Figure 8-14. LLM rerankers operate as part of a search pipeline with the goal of reordering a number of shortlisted search results by relevance.

Retrieval Evaluation Metrics



- Semantic search is evaluated using metrics from the Information Retrieval (IR) field. Let's discuss one of these popular metrics: mean average precision (MAP). Evaluating search systems needs three major components: a text archive, a set of queries, and relevance judgments indicating which documents are relevant for each query. We see these components in Figure 8-16.

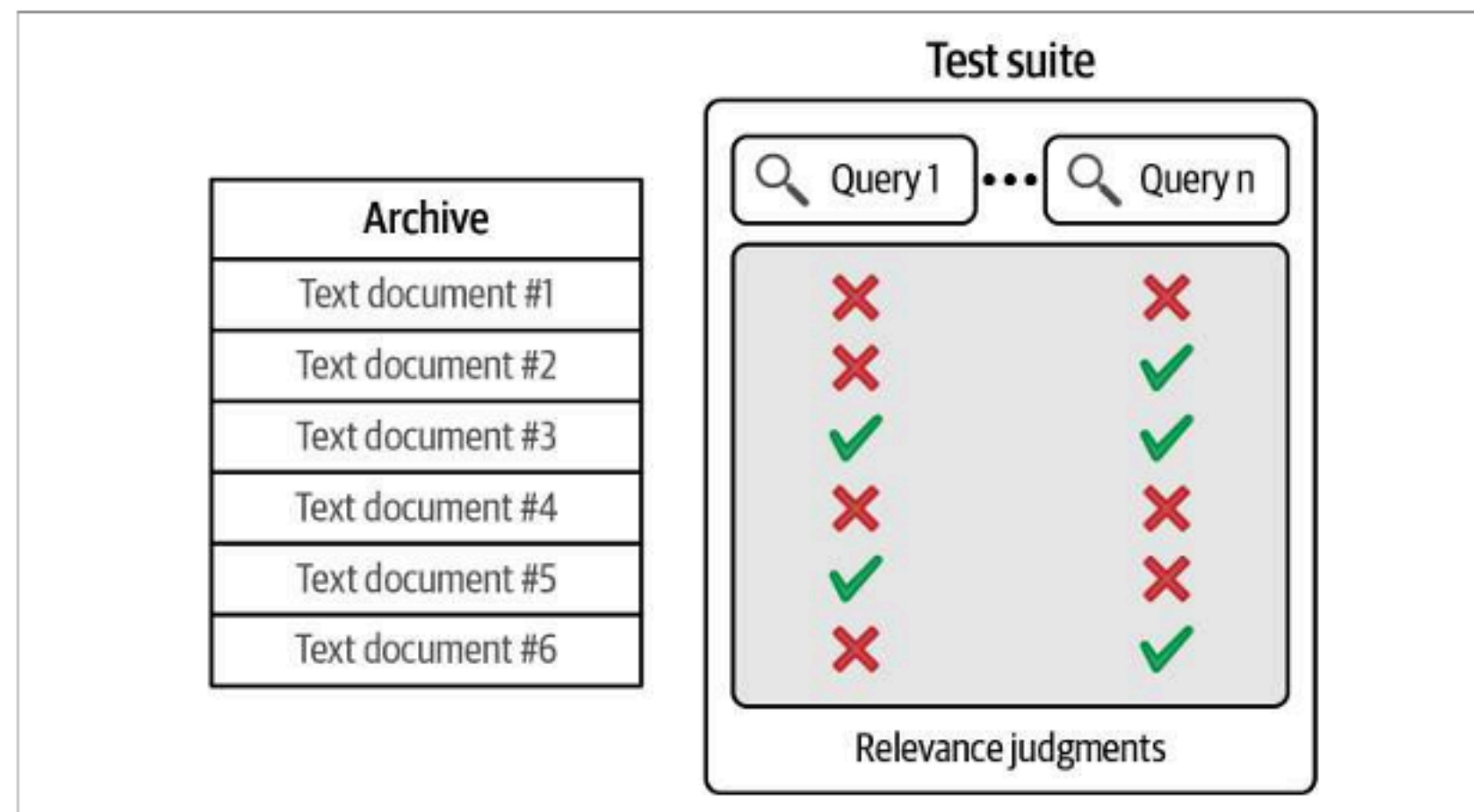


Figure 8-16. To evaluate search systems, we need a test suite including queries and relevance judgments indicating which documents in our archive are relevant for each query.

- Using this test suite, we can proceed to explore evaluating search systems. Let's start with a simple example. Let's assume we pass query 1 to two different search systems. And get two sets of results. Say we limit the number of results to three, as we can see in Figure 8-17.

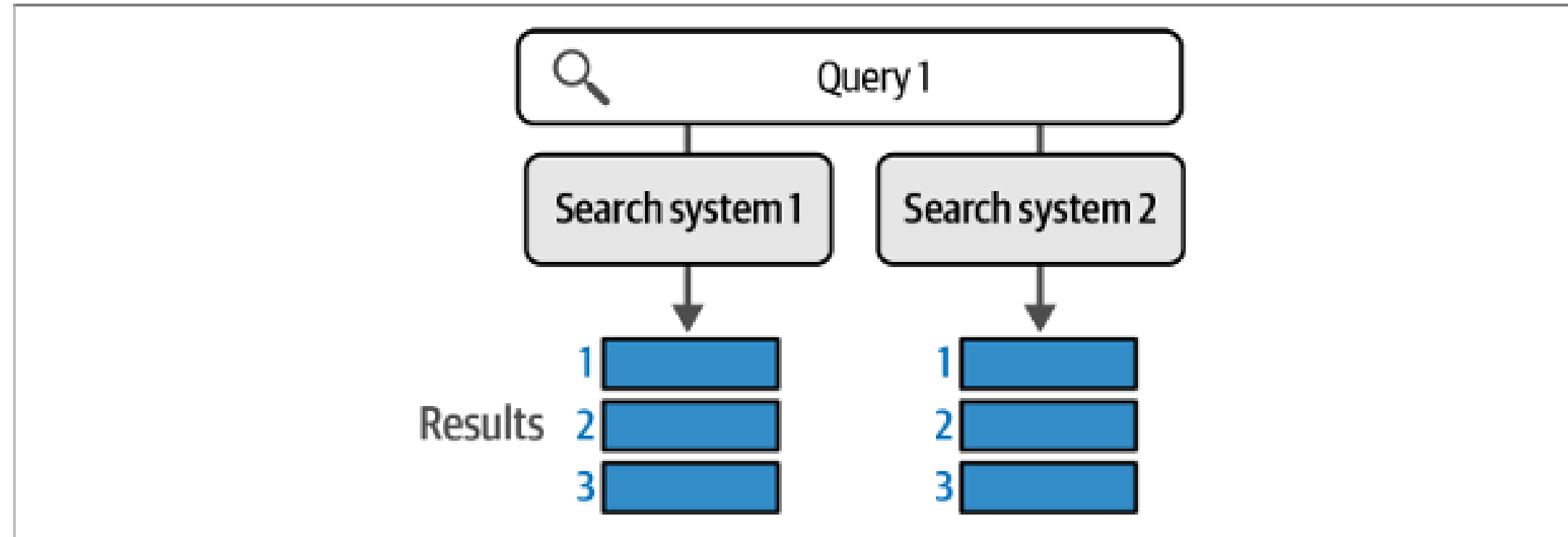


Figure 8-17. To compare two search systems, we pass the same query from our test suite to both systems and look at their top results.

- To tell which is a better system, we turn to the relevance judgments that we have for the query. Figure 8-18 shows which of the returned results are relevant.

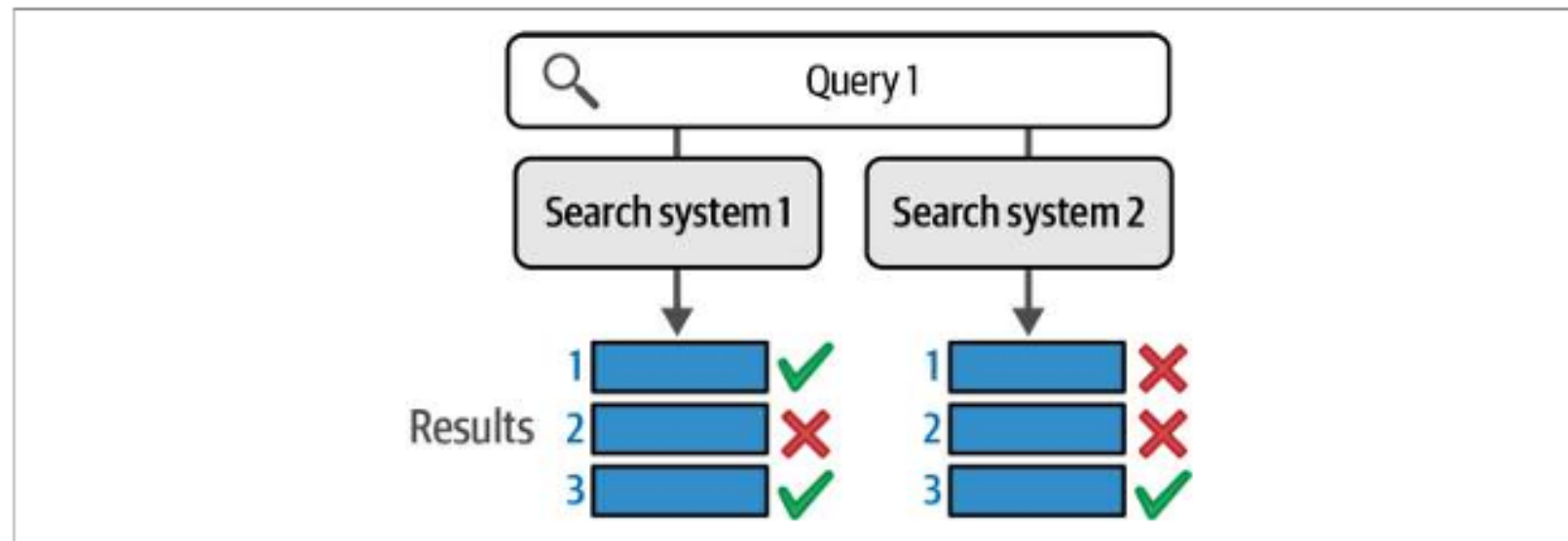


Figure 8-18. Looking at the relevance judgments from our test suite, we can see that system 1 did a better job than system 2.

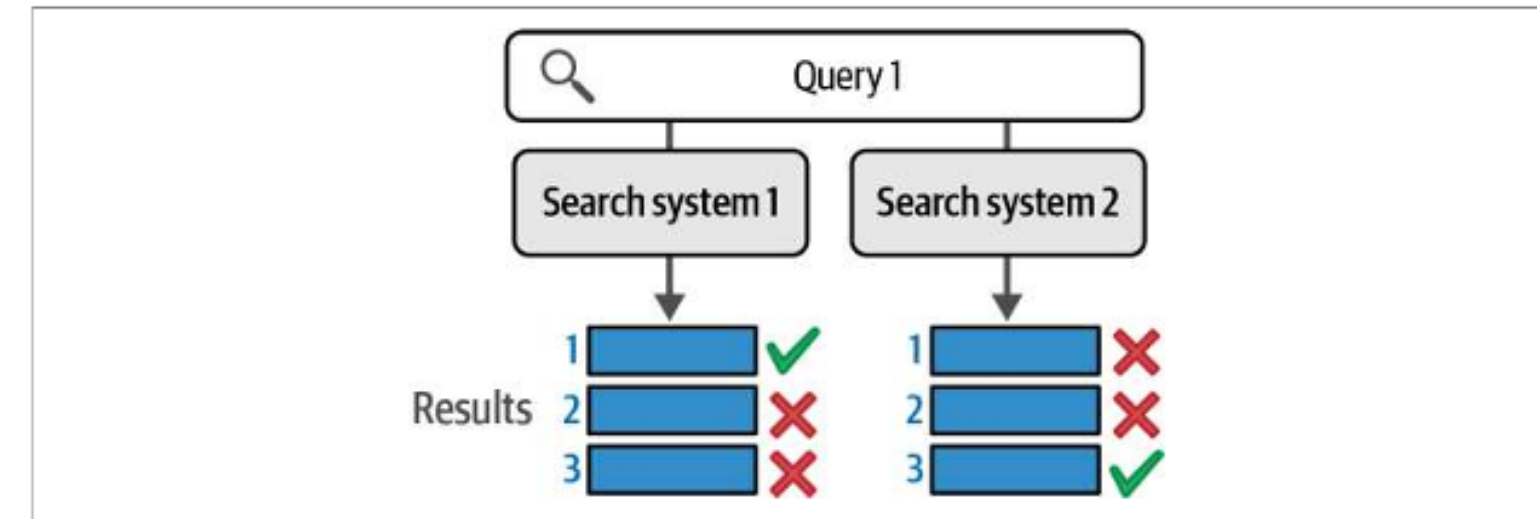


Figure 8-19. We need a scoring system that rewards system 1 for assigning a high position to a relevant result—even though both systems retrieved only one relevant result in their top three results.

- This shows us a clear case where system 1 is better than system 2. Intuitively, we may just count how many relevant results each system retrieved. System 1 got two out of three correct, and system 2 got only one out of three correct. But what about a case like Figure 8-19 where both systems only get one relevant result out of three, but they're in different positions?



- In this case, we can intuit that system 1 did a better job than system 2 because the result in the first position (the most important position) is correct. But how can we assign a number or score to how much better that result is? Mean average precision is a measure that is able to quantify this distinction.
- One common way to assign numeric scores in this scenario is average precision, which evaluates system 1's result for the query to be 1 and system 2's to be 0.3. So let's see how average precision is calculated to evaluate one set of results, and then how it's aggregated to evaluate a system across all the queries in the test suite.

Scoring a single query with average precision

- To score a search system on this query, we can focus on scoring the relevant documents. Let's start by looking at a query that only has one relevant document in the test suite. The first one is easy: the search system placed the relevant result (the only available one for this query) at the top. This gets the system the perfect score of . Figure 8-20 shows this calculation: looking at the first position, we have a relevant result leading to a precision at position 1 of 1.0 (calculated as the number of relevant results at position 1, divided by the position we're currently looking at).

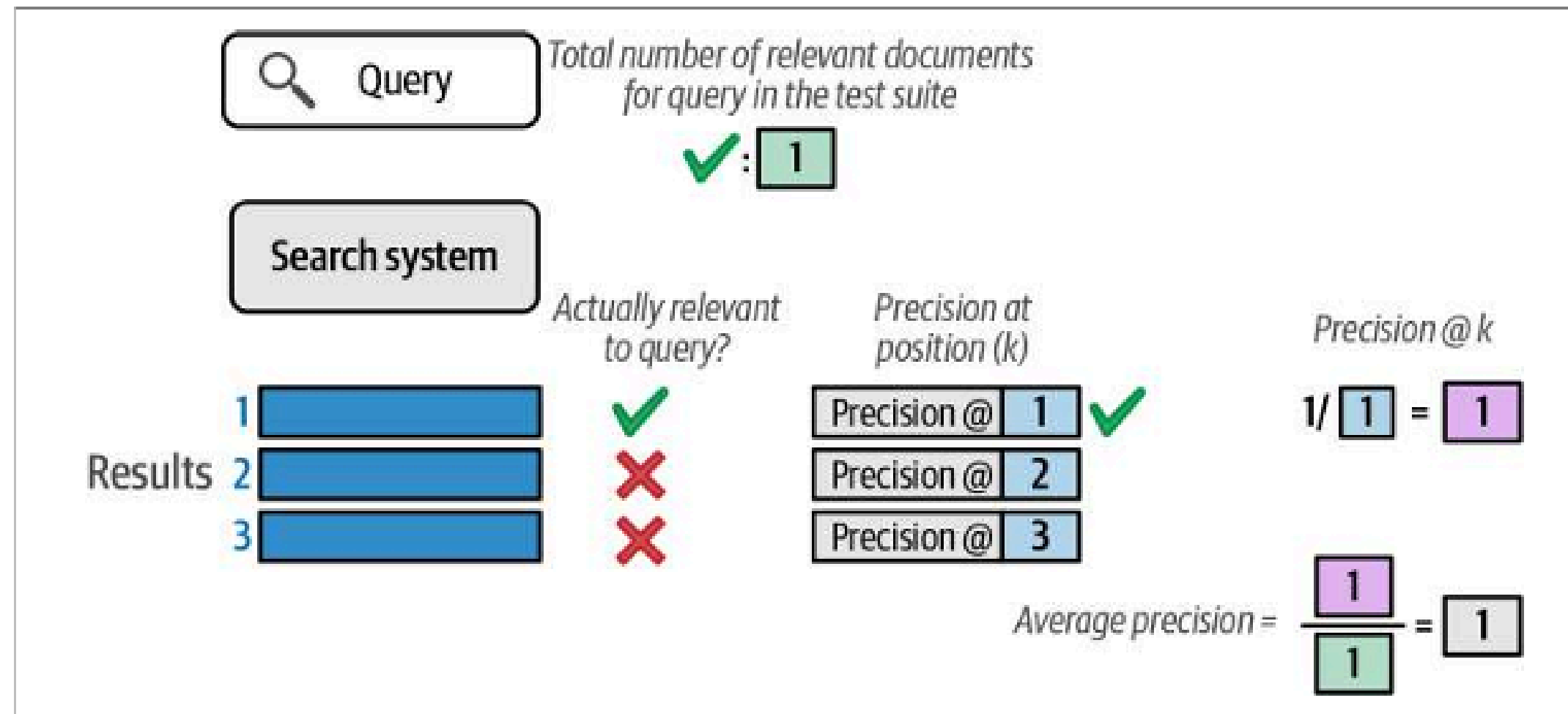


Figure 8-20. To calculate mean average precision, we start by calculating precision at each position, starting with position 1.

- Since we're only scoring relevant documents we can ignore the scores of nonrelevant documents and stop our calculation here. What if the system actually placed the only relevant result at the third position, however? How would that affect the score? Figure 8-21 shows how that results in a penalty.

- Since we're only scoring relevant documents we can ignore the scores of nonrelevant documents and stop our calculation here. What if the system actually placed the only relevant result at the third position, however? How would that affect the score? Figure 8-21 shows how that results in a penalty.

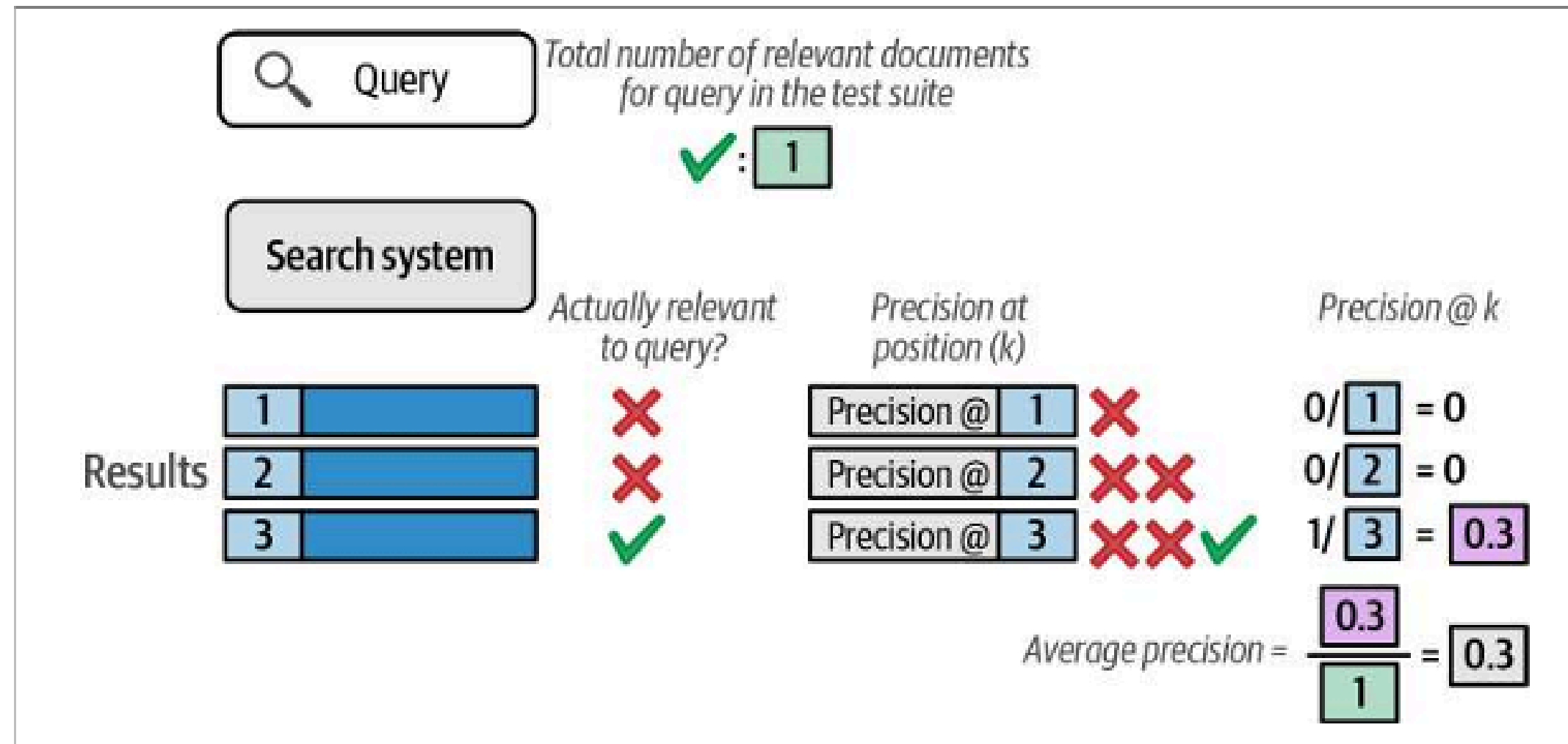


Figure 8-21. If the system places nonrelevant documents ahead of a relevant document, its precision score is penalized.

- Let's now look at a query with more than one relevant document. Figure 8-22 shows that calculation and how averaging now comes into the picture.

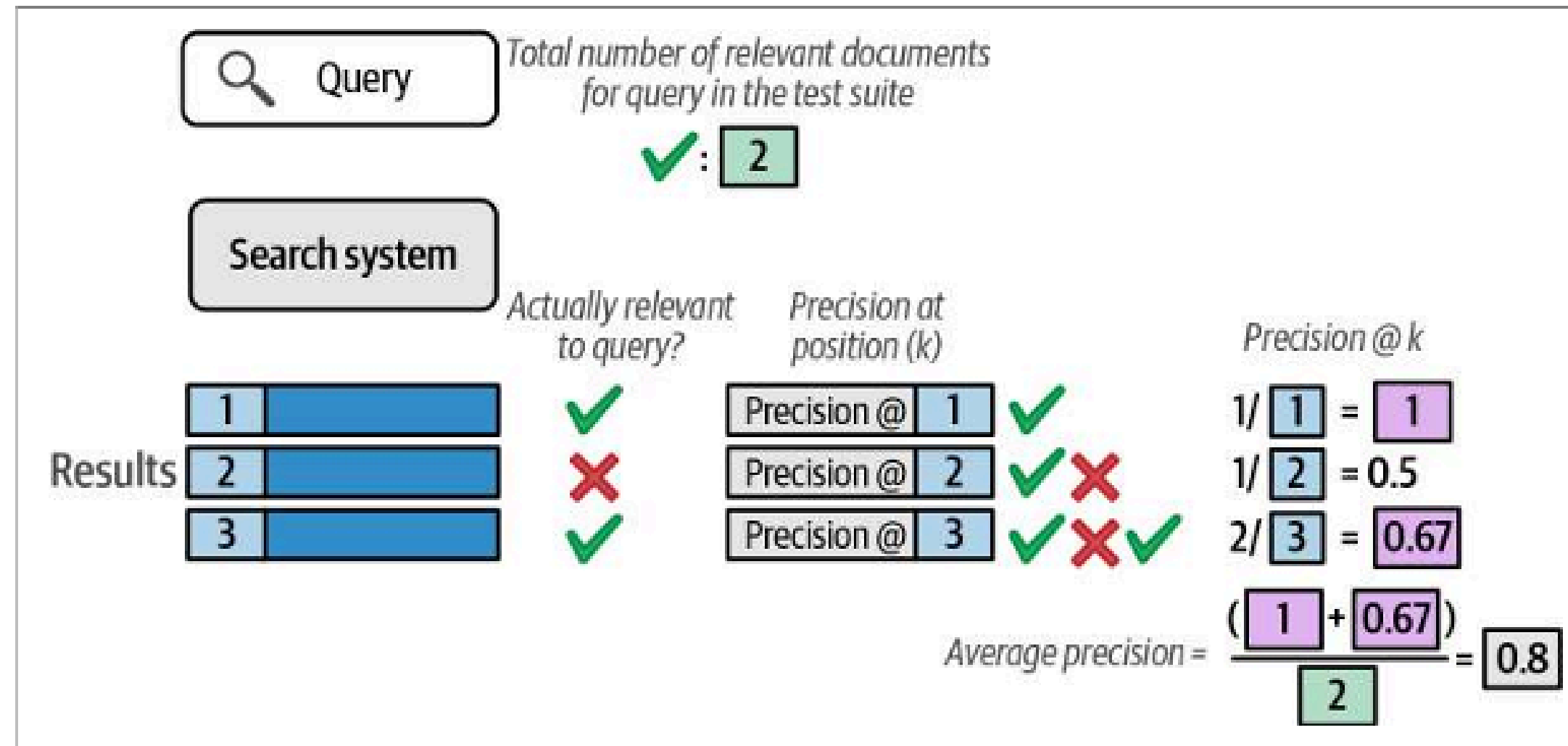


Figure 8-22. Average precision of a document with multiple relevant documents considers the precision at k results of all the relevant documents.

Scoring across multiple queries with mean average precision

- Now that we're familiar with precision at k and average precision, we can extend this knowledge to a metric that can score a search system against all the queries in our test suite. That metric is called mean average precision. Figure 8-23 shows how to calculate this metric by taking the mean of the average precisions of each query.

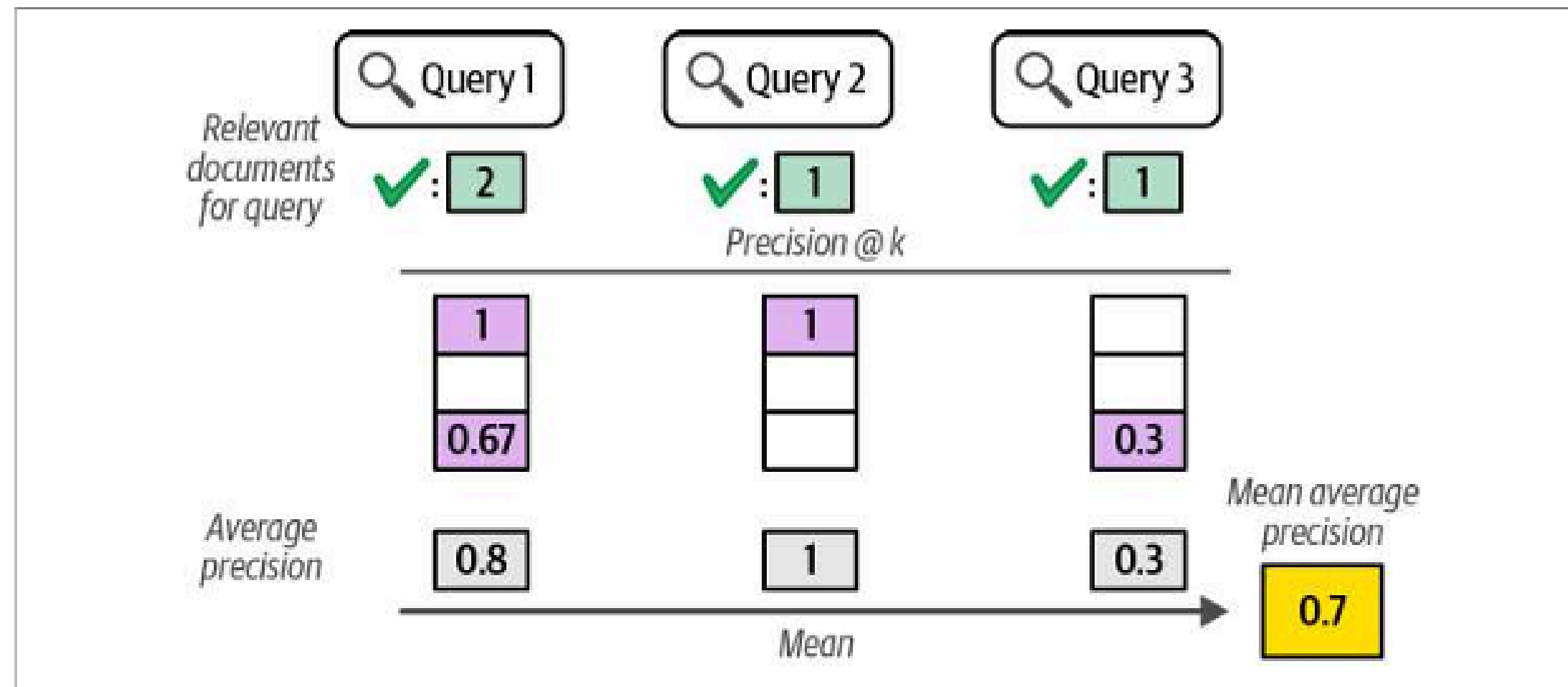


Figure 8-23. The mean average precision takes into consideration the average precision score of a system for every query in the test suite. By averaging them, it produces a single metric that we can use to compare a search system against another.



- You may be wondering why the same operation is called “mean” and “average.” It’s likely an aesthetic choice because MAP sounds better than average average precision.
- Now we have a single metric that we can use to compare different systems. If you want to learn more about evaluation metrics, see the “Evaluation in Information Retrieval” chapter of Introduction to Information Retrieval (Cambridge University Press) by Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze.
- In addition to mean average precision, another metric commonly used for search systems is normalized discounted cumulative gain (nDCG), which is more nuanced in that the relevance of documents is not binary (relevant versus not relevant) and one document can be labeled as more relevant than another in the test suite and scoring mechanism.

COURSE 1

LARGE LANGUAGE MODELS (LLMs) & AGENTIC AI

